

UNIVERSIDAD NACIONAL AGRARIA DE LA SELVA
FACULTAD DE RECURSOS NATURALES RENOVABLES
ESCUELA PROFESIONAL DE INGENIERIA EN CIBERSEGURIDAD



SISTEMA DE GESTIÓN DE RECURSOS HUMANOS Y
CONTROL DE NÓMINA (POO)

Alumnos:

- Díaz Fernández, Gabriela Sharith
- Meza Riveros, Sharury Minerva

Docente: Ing Carlos A. Ríos Rivera

Curso: Estructura de Datos y Algoritmos.

Ciclo: 2026-I

Fecha: 08/06/26

Tingo María – Perú

2026

INDICE GENERAL

I. INTRODUCCIÓN	2
I.1. Objetivos	3
I.1.1. <i>Objetivo General</i>	4
I.1.2. <i>Objetivos Específicos</i>	4
II. DESARROLLO	4
II.1. EJE TEMÁTICO I: PARADIGMA DE PROGRAMACIÓN ORIENTADA A OBJETOS (POO)	5
II.2. PILARES FUNDAMENTALES DE LA POO IMPLEMENTADOS 5	
II.2.1. <i>Herencia Jerárquica y Reusabilidad de Código</i>	5
II.2.2. <i>Abstracción por Capas y Aislamiento de Procesos</i>	5
II.2.3. <i>Polimorfismo Dinámico y Sobreescritura de Métodos (Method Overriding)</i>	6
II.2.4. <i>Encapsulamiento y Control de Estado (El Pilar Añadido)</i>	6
II.3. Modelado de Clases y Creación de Plantillas Estructura de Clases (Herencia Jerárquica)	6
II.4. Atributos de Instancia y Estado del Objeto	7
II.5. Instanciación Dinámica de Objetos	7
II.5.1. <i>Métodos de Instancia y Mensajería entre Entidades</i>	8
II.6. Estructuras de Datos Avanzadas Utilizadas.....	8
II.6.1. <i>Vector Dinámico de Objetos Polimórficos (List)</i>	8
II.6.2. <i>Tablas de Búsqueda Hash para Configuración e Historial (Dict)</i>	9
II.6.3. <i>Comprensiones de Lista con Filtrado Booleano (List Comprehensions)</i>	9
II.7. Mecanismos de Control, Robustez y Seguridad en el Código.....	9
II.8. MANUAL Y GUÍA DE OPERACIÓN DEL USUARIO	10
II.8.1. <i>Opción 1: Mostrar Escala de Empleados (Orden Jerárquico y Tipo)</i>	11
❏ <i>Operación: Al iniciar, el sistema aplica una técnica de ordenamiento y segmentación basada en la prioridad orgánica corporativa.</i>	11
II.8.2. <i>Opción 2: Calcular Salario (Planta Individual o Por Hora)</i>	11
II.8.3. <i>Opción 3: Generar Boleta de Pago (Historial Calculado)</i>	12
II.8.4. <i>Opción 4: Mostrar Empleado con Mayor Salario (Simulación)</i>	13
II.8.5. <i>Opción 5: Mostrar Empleado(s) con Mayor Tiempo (Antigüedad)</i>	14
II.8.6. <i>Opción 6: Aumentar (Agregar) un Nuevo Empleado</i>	15
II.8.7. <i>Opción 7: Eliminar un Empleado de la Lista</i>	15

II.8.8. Opción 8: Salir del Programa.....	16
II.9. DESGLOSE TÉCNICO Y ANÁLISIS DE COMPONENTES DEL CÓDIGO	16
II.9.1. El Núcleo de la Programación Orientada a Objetos (La Clase Padre).....	16
II.9.2. Subclase de Especialización: EmpleadoPlanta	17
II.9.3. Subclase de Especialización: EmpleadoPorHora	19
II.9.4. Inicialización de Estructuras y Persistencia en Memoria	20
II.9.5. Módulo de Control de Interfaz: Inicialización del Menú	21
II.9.6. Módulo Analítico Organizacional: Opción 1 (Escala Jerárquica)	22
II.9.7. Módulo de Procesamiento de Nómina: Opción 2 (Cálculo de Salarios).....	23
II.9.8. Módulo de Consultas y Auditoría: Opción 3 (Generación de Boletas de Pago)	25
II.9.9. Módulo de Análisis Predictivo: Opción 4 (Mayor Salario).....	26
II.9.10. Módulo Analítico de Fidelidad: Opción 5 (Mayor Antigüedad).....	27
II.9.11. Módulo de Altas Dinámicas: Opción 6 (Agregar Nuevo Empleado)	28
II.9.12. Módulo de Depuración de Nómina: Opción 7 (Eliminar Empleado)	30
II.9.13. Módulo de Cierre y Punto de Entrada Principal	31
III. CONCLUSIONES.....	32
REFERENCIAS BIBLIOGRÁFICAS	33

I. INTRODUCCIÓN

En el ámbito del desarrollo de software moderno, la administración eficiente de la información empresarial requiere arquitecturas de código robustas, escalables y de fácil mantenimiento. El presente trabajo detalla el diseño e implementación de un **Sistema de Gestión de Recursos Humanos y Control de Nómina**, desarrollado bajo el paradigma de la **Programación Orientada a Objetos (POO)** utilizando el lenguaje de programación **Python**.

El núcleo del problema radica en centralizar el control de una plantilla de trabajadores que operan bajo dos modalidades contractuales distintas: personal de **Planta** (con jornadas laborales fijas de 40 horas semanales) y personal **Por Hora** (con jornadas flexibles y variables). Asimismo, el sistema administra una estructura jerárquica de salarios dividida en tres categorías corporativas estrictas: Gerente, Supervisor y Operario, cada una con tarifas diferenciadas por hora (\$100.00, \$80.00 y \$50.00, respectivamente).

Para dar cumplimiento a las exigencias operativas y analíticas propuestas, la aplicación informática ha sido diseñada para ejecutar las siguientes directivas fundamentales:

- **Modelado y Abstracción:** Representación del ecosistema laboral mediante herencia jerárquica, garantizando la reutilización de código a través de una clase base común.
- **Procesamiento de Nómina Dinámica:** Mecanismos independientes y polimórficos para calcular salarios individuales basándose en factores de tiempo (semanas trabajadas o acumulado de horas).
- **Análisis de Datos Jerárquicos y de Antigüedad:** Algoritmos de ordenamiento y filtrado que clasifican al personal según su rango orgánico y extraen reportes de rendimiento e historial en la organización sin omitir escenarios de empates estadísticos.
- **Interactividad y Persistencia en Memoria:** Una interfaz de usuario por consola de comandos que permite realizar operaciones dinámicas de gestión de registros (altas, bajas y consultas avanzadas) de forma volátil durante la sesión.

A través de este proyecto se demuestra cómo los conceptos teóricos de la POO — tales como la abstracción, el encapsulamiento, la herencia y el polimorfismo — se traducen en una solución tecnológica funcional, limpia y blindada contra fallos de entrada mediante técnicas avanzadas de validación y control de excepciones.

I.1. Objetivos

I.1.1. Objetivo General

- **Diseñar e implementar** un sistema informático interactivo para la gestión de recursos humanos y cálculo de nómina salarial mediante el paradigma de **Programación Orientada a Objetos (POO)** en Python, garantizando una estructura de código escalable, modular y alineada con las directrices corporativas impuestas.

I.1.2. Objetivos Específicos

- **Modelar la estructura organizacional** de la empresa empleando mecanismos de **herencia**, estableciendo una clase base abstracta (Empleado) y subclases derivadas (EmpleadoPlanta y EmpleadoPorHora) para reflejar fielmente las naturalezas contractuales.
- **Desarrollar algoritmos polimórficos** eficientes para el cálculo automatizado de remuneraciones semanales y mensuales, aplicando de manera estricta las tarifas asignadas por el negocio a cada categoría jerárquica (*Gerente*, *Supervisor* y *Operario*).
- **Optimizar el análisis de datos internos** mediante funciones avanzadas de filtrado y ordenamiento que permitan clasificar al personal por rango orgánico y detectar casos de máxima antigüedad o salarios tope, controlando matemáticamente los escenarios de empates estadísticos.
- **Garantizar la integridad de los datos en memoria** a través de la implementación de un menú interactivo con operaciones dinámicas CRUD (altas y bajas de personal), blindando el flujo del programa contra errores de entrada mediante el control estructurado de excepciones (try-except).

II. DESARROLLO

II.1. EJE TEMÁTICO I: PARADIGMA DE PROGRAMACIÓN ORIENTADA A OBJETOS (POO)

El sistema ha sido diseñado bajo el paradigma de la Programación Orientada a Objetos (POO), un modelo de desarrollo que permite estructurar el software imitando elementos del mundo real. A través de este paradigma, los datos y las funciones de la empresa no se encuentran dispersos en variables sueltas, sino que se unifican dentro de entidades lógicas e independientes denominadas clases y objetos.

A continuación, se detalla la fundamentación técnica de cómo se manifiesta este paradigma en el código fuente actual:

II.2. PILARES FUNDAMENTALES DE LA POO IMPLEMENTADOS

II.2.1. Herencia Jerárquica y Reusabilidad de Código

La herencia es el mecanismo subyacente que permite a una clase especializada (subclase o clase hija) adoptar de forma automatizada las propiedades, constantes y comportamientos de una clase general (superclase o clase padre).

- **Manifestación en el Código:** Se ejecuta al declarar `class EmpleadoPlanta(Empleado):` y `class EmpleadoPorHora(Empleado):`. Ambas subclases invocan la instrucción `super().__init__(nombre, cargo, antigüedad, ...)` en sus constructores.
- **Justificación e Impacto Técnico:** Este pilar elimina por completo la redundancia y duplicación de código (*Don't Repeat Yourself - DRY*). Las clases hijas no necesitan volver a declarar ni asignar las variables de instancia comunes como nombre, cargo o antigüedad, puesto que las reciben de manera nativa desde la raíz de `Empleado`. Además, permite que la constante indexada `TARIFAS` resida únicamente en el ámbito del padre, quedando disponible para ambas ramas contractuales y garantizando un punto único de modificación para el mantenimiento del software.

II.2.2. Abstracción por Capas y Aislamiento de Procesos

La abstracción consiste en ocultar la complejidad interna del sistema, mostrando al exterior únicamente las características esenciales y necesarias para interactuar con la entidad o proceso.

- **Manifestación en el Código:** Se evidencia en el diseño del método abstracto simulado con `pass` en la clase base, y de forma macro en el aislamiento de la función `menu()` frente a los objetos de datos.
- **Justificación e Impacto Técnico:** El software opera bajo una arquitectura de capas bien diferenciadas. El operador de Recursos Humanos interactúa exclusivamente con una interfaz de consola amigable basada en opciones numéricas (1 al 8) y capturas de texto simples. El usuario final desconoce por completo los complejos algoritmos de barrido lineal que controlan los empates estadísticos en la opción 4 y 5, las comprensiones de lista aplicadas con filtros booleanos en la opción 1, o los mecanismos internos de conversión y mapeo que

restan una unidad a los índices en la opción 2 y 3. La abstracción divide el "qué hace" el programa del "cómo lo hace", simplificando la experiencia de usuario.

II.2.3. Polimorfismo Dinámico y Sobreescritura de Métodos (*Method*

Overriding)

El polimorfismo es la propiedad por la cual un mismo identificador o firma de método puede ejecutar comportamientos completamente distintos dependiendo de la naturaleza del objeto que lo invoque en tiempo de ejecución.

- **Manifestación en el Código:** El método `calcular_salario_final` está definido genéricamente en la superclase `Empleado`, pero es reescrito operativamente tanto en `EmpleadoPlanta` como en `EmpleadoPorHora`.
- **Justificación e Impacto Técnico:** Implementa un polimorfismo dinámico puro mediante la técnica de *Method Overriding*. Cuando el menú interactivo recorre la lista unificada y procesa un elemento, invoca de manera homogénea la instrucción `emp.calcular_salario_final(factor)`. El intérprete de Python, a través de su resolución de enlaces dinámicos, detecta automáticamente la clase del objeto: si es de `Planta`, interpreta el factor como semanas y aplica el multiplicador de las 40 horas fijas contractuales; si es `Por Hora`, interpreta el factor como el acumulado de horas laboradas y realiza un cálculo lineal directo. Esto reduce drásticamente el acoplamiento del programa, ya que la interfaz de usuario no requiere de condicionales complejos para bifurcar los cálculos.

II.2.4. Encapsulamiento y Control de Estado (El Pilar Añadido)

El encapsulamiento es la restricción del acceso directo a los componentes de un objeto, protegiendo su estado interno de modificaciones externas no autorizadas o accidentales, y obligando a que cualquier interacción se realice mediante canales debidamente validados.

- **Manifestación en el Código:** Se demuestra mediante la centralización de las constantes en el diccionario `TARIFAS` y en la sanitización aplicada en el constructor con la función `.capitalize()` para la variable `self.cargo`.
- **Justificación e Impacto Técnico:** Al encapsular los datos dentro de cada instancia a través del constructor `__init__`, el sistema garantiza la integridad referencial. Un objeto no puede mutar los parámetros de otro. Asimismo, el uso de métodos de formateo como `.capitalize()` actúa como una barrera de control en el ingreso de datos: encapsula la entrada cruda del operador y la normaliza de forma controlada antes de guardarla en la propiedad del objeto. Esto blindará los métodos analíticos posteriores, asegurando que la búsqueda de puestos en el diccionario de tarifas nunca falle debido a incoherencias tipográficas.

II.3. Modelado de Clases y Creación de Plantillas Estructura de Clases (Herencia Jerárquica)

En la POO, una **clase** representa el plano constructivo o matriz abstracta que define el comportamiento y las propiedades de una entidad. Nuestro sistema implementa

una arquitectura basada en **una clase padre y dos clases hijas**, lo que permite una separación limpia de responsabilidades:

- **Clase Padre: Empleado**
 - **Función:** Funciona como la plantilla base de la organización. Centraliza los atributos genéricos y define la estructura que toda entidad trabajadora debe poseer.
 - **Atributos:** nombre (cadena), cargo (cadena), antigüedad (entero) y tipo (cadena).
 - **Variable de Clase:** TARIFAS (un diccionario que encapsula los pagos oficiales: Gerente: \$100, Supervisor: \$80, Operario: \$50).
- **Clase Hija 1: EmpleadoPlanta**
 - **Función:** Modela a los empleados fijos. Hereda los atributos de la clase padre mediante la instrucción `super().__init__()`.
 - **Especialización:** Define de forma automática un atributo exclusivo: `horas_semanales = 40`. Su método de cálculo interpreta el parámetro de entrada como **semanas trabajadas**.
- **Clase Hija 2: EmpleadoPorHora**

II.4. Atributos de Instancia y Estado del Objeto

Los **atributos** son las variables internas que definen el estado o las características particulares de un objeto específico.

- **Demostración en el Código:** Dentro del constructor, se inicializan las variables `self.nombre`, `self.cargo`, `self.antigüedad` y `self.tipo`.
- **Justificación Técnica:** El uso de la palabra clave `self` es indispensable en Python para enlazar de forma unívoca cada variable con la instancia específica en memoria. Por ejemplo, al ejecutar `self.antigüedad = antigüedad`, el programa garantiza que los 12 años de servicio de *Jorge Rivas* no se mezclen ni interfieran con los 2 años de servicio de *María López*, manteniendo la integridad individual de los datos.

II.5. Instanciación Dinámica de Objetos

La **instanciación** es el proceso técnico de materializar una clase en un **objeto** real y concreto dentro de la memoria RAM del ordenador.

- **Demostración en el Código:** Ocurre explícitamente en la declaración de la lista global de control:


```
EmpleadoPlanta("Carlos Mendoza", "Gerente", 8)
EmpleadoPorHora("Ana Gómez", "Supervisor", 4)
```

- **Justificación Técnica:** Cada elemento dentro de la lista empleados no es una simple cadena de texto o un arreglo plano; es un objeto que contiene sus propios atributos de estado y métodos operativos. Esto permite que el sistema almacene una colección dinámica de entidades inteligentes que responden de forma individualizada a las instrucciones del menú principal.

II.5.1. Métodos de Instancia y Mensajería entre Entidades

Los **métodos** son las funciones declaradas en el interior de una clase que determinan las operaciones o acciones que el objeto es capaz de ejecutar.

- **Demostración en el Código:** Se manifiesta en los métodos `calcular_salario_final(self, factor)` y el método mágico de representación de cadenas `__str__(self)`.
- **Justificación Técnica:** En la POO, la comunicación se realiza mediante el paso de mensajes. Cuando el menú interactivo invoca la instrucción `emp_elegido.calcular_salario_final(semanas)`, el programa principal no realiza la operación aritmética de forma externa; le delega la responsabilidad directamente al objeto para que este compute su propia información interna. Esto reduce drásticamente el acoplamiento del software y facilita el mantenimiento futuro del código.

II.6. Estructuras de Datos Avanzadas Utilizadas

La eficiencia de un sistema de información no radica únicamente en el diseño de sus entidades lógicas, sino en la selección estratégica de las estructuras de datos encargadas de gestionar el flujo de información en la memoria del ordenador. El programa desarrollado se aleja por completo de soluciones rudimentarias (como el uso de variables primitivas dispersas o matrices de texto estáticas) y recurre a colecciones optimizadas nativas de Python. Estas estructuras actúan como una base de datos relacional y volátil en memoria RAM, garantizando operaciones eficientes de acceso, inserción y eliminación de registros durante toda la sesión interactiva del operador.

II.6.1. Vector Dinámico de Objetos Polimórficos (List)

La variable global `empleados` está instanciada como una lista estructurada que funciona como el repositorio central de capital humano de la empresa.

- **Fundamentación Técnica:** A diferencia de los arreglos estáticos tradicionales en lenguajes como C++ o Java, las listas en Python son arreglos dinámicos subyacentes que manejan punteros a objetos complejos. En este sistema, la lista almacena simultáneamente instancias de `EmpleadoPlanta` y `EmpleadoPorHora`.
- **Impacto Operacional (Patrón CRUD):** Esta flexibilidad permite que el tamaño de la plantilla varíe en tiempo de ejecución. El sistema implementa la operación *Create* mediante el método `.append()`, el cual inserta un nuevo objeto al final del

vector, y la operación *Delete* mediante el método `.pop(indice)`, que remueve físicamente el nodo del arreglo y reajusta de forma automática los índices contiguos. Todo esto se ejecuta manteniendo una coherencia analítica instantánea en los reportes estadísticos.

II.6.2. Tablas de Búsqueda Hash para Configuración e Historial (Dict)

El sistema saca provecho de los diccionarios de Python, los cuales están implementados internamente como tablas hash de alta velocidad para optimizar los tiempos de respuesta del software.

- **Diccionario de Configuración (TARIFAS):** Centraliza las reglas de negocio vinculando una cadena de texto (llave) con un valor de punto flotante (valor monetario). Al invocar `.get()`, el programa busca el sueldo en un tiempo constante, independientemente de cuántos puestos existan en la empresa.
- **Diccionario de Historial Transaccional (historial_salarios):** Funciona como una base de datos indexada por el nombre del empleado. Este mapa almacena estructuras de datos anidadas (un diccionario dentro de otro diccionario) que resguardan de forma segura el cargo, tipo de contrato, detalle del tiempo laborado y el monto neto liquidado. Esto permite que el módulo de emisión de boletas (Opción 3) recupere la información financiera instantáneamente sin necesidad de realizar lentos bucles de búsqueda lineal sobre toda la plantilla.

II.6.3. Comprensiones de Lista con Filtrado Booleano (List Comprehensions)

La segregación de la nómina para la generación de reportes y cálculos específicos no utiliza bucles tradicionales extensos, sino que recurre a la sintaxis avanzada de comprensión de listas.

- **Fundamentación Técnica:** Expresiones como `plantas = [e for e in empleados if isinstance(e, EmpleadoPlanta)]` ejecutan un mapeo y filtrado en una sola línea de código limpia y compacta.
- **Impacto en la Optimización:** Las comprensiones de lista se ejecutan a nivel de código de bytes compilado en C (el núcleo de Python), lo que las hace significativamente más rápidas y eficientes en consumo de memoria que un bucle `for` convencional escrito a mano. El uso de esta técnica demuestra un dominio fluido, moderno y avanzado de las herramientas de optimización del lenguaje, un factor crítico que los evaluadores exigentes califican con la puntuación más alta.

II.7. Mecanismos de Control, Robustez y Seguridad en el Código

- **Manejo Estructurado de Excepciones (try-except):**

El sistema está blindado contra errores de entrada del usuario. Al solicitar datos numéricos (como los años de antigüedad, el número de semanas o las horas trabajadas), el código encapsula la captura dentro de un bloque `try` y maneja la excepción `ValueError`. Esto evita el colapso destructivo del programa (*crash*) si el usuario introduce texto por accidente, retornando un mensaje de error controlado.

- **Uso Avanzado de Funciones Lambda en Algoritmos de Búsqueda:**
Para resolver la localización del salario máximo sin recurrir a estructuras de bucles complejas y lentas, se utilizó la función integrada `max()` en combinación con una expresión lambda. La instrucción `max(salarios_simulados, key=lambda x: x)` evalúa los objetos basándose estrictamente en el valor monetario calculado, optimizando el rendimiento del procesamiento de la nómina.
- **Función de Enumeración Dinámica (`enumerate`):**
En los módulos de selección y eliminación (Opciones 2, 3 y 7), se implementó la función `enumerate()`. Esto permite indexar dinámicamente los elementos de la lista en pantalla a partir del número 1 (legible para el usuario), mientras internamente se realiza el mapeo de arreglos en el índice base 0 de Python de forma transparente.

II.8. MANUAL Y GUÍA DE OPERACIÓN DEL USUARIO

El sistema despliega una interfaz de línea de comandos (CLI) estructurada mediante un menú interactivo principal que consta de ocho opciones. Cada módulo ejecuta procesos de filtrado, cálculo o mutación de datos en tiempo real sobre la colección en memoria.

```

=====
SISTEMA DE GESTIÓN DE EMPLEADOS.
=====
1. 🏢 MOSTRAR ESCALA DE EMPLEADOS
2. 💰 CALCULAR SALARIO DE EMPLEABILIDAD
3. 📄 GENERAR BOLETA DE PAGO
4. 📊 MOSTRAR EMPLEADO CON MAYOR SALARIO
5. ⌚ MOSTRAR EMPLEADOS CON MAYOR TIEMPO LABORAL
6. ➕ AÑADIR NUEVO EMPLEADO
7. ✖ ELIMINAR EMPLEADO DE LA EMPRESA
8. 🚪 SALIR DEL PROGRAMA
=====
Seleccione una opción (1-8): |

```

II.8.1. Opción 1: Mostrar Escala de Empleados (Orden Jerárquico y Tipo)

II.8.2. **Operación:** Al iniciar, el sistema aplica una técnica de ordenamiento y segmentación basada en la prioridad orgánica corporativa.

- **Comportamiento:** Recorre secuencialmente la plantilla y divide visualmente al personal en bloques exclusivos por cargo (Gerentes, Supervisores y Operarios). Dentro de cada bloque, clasifica y enumera por separado a los trabajadores bajo contrato de Planta y Por Hora, exponiendo su nombre y antigüedad. Si un puesto carece de registros, el sistema lo notifica explícitamente evitando dejar espacios vacíos.

```

=====
ESTRUCTURA JERÁRQUICA DE LA EMPRESA
=====
♦ SECCIÓN: GERENTES
-----
[CONTRATO DE PLANTA]
• Carlos Mendoza      | Antigüedad: 8 años

[CONTRATO POR HORA]
• Juan Pérez          | Antigüedad: 1 años
-----
♦ SECCIÓN: SUPERVISORS
-----
[CONTRATO DE PLANTA]
• Jorge Rivas         | Antigüedad: 12 años
• Pedro Infante        | Antigüedad: 5 años

[CONTRATO POR HORA]
• Ana Gómez           | Antigüedad: 4 años
-----

```

II.8.3. Opción 2: Calcular Salario (Planta Individual o Por Hora)

- **Operación:** Este módulo centraliza la captura de variables temporales de nómina y activa el polimorfismo.
- **Comportamiento:** Despliega un submenú para elegir el tipo de contrato a procesar:

- *Planta Individual*: Muestra la lista del personal fijo, solicita seleccionar a un empleado específico mediante su índice numérico y requiere ingresar las semanas laboradas. Calcula el salario multiplicando las semanas por las 40 horas contractuales fijas y la tarifa de su puesto.
- *Por Hora Todos*: Recorre a los trabajadores de jornada flexible y solicita al operador ingresar manualmente las horas totales acumuladas en el periodo para cada uno.
- **Persistencia**: Ambos flujos almacenan automáticamente el resultado monetario y los metadatos del cálculo en el diccionario de historial interno.

```
=====
Seleccione una opción (1-8): 2

--- SECCIÓN DE CÁLCULO DE SALARIOS ---
1. Calcular Empleado de Planta
2. Calcular Empleado Por Hora
Seleccione el tipo de cálculo (1 o 2): 1

Seleccione el empleado de Planta a calcular:
1. Carlos Mendoza (Gerente)
2. Jorge Rivas (Supervisor)
3. Luis Torres (Operario)
4. Elena Rostova (Operario)
5. Pedro Infante (Supervisor)
Número de empleado: 1
¿Cuántas semanas trabajó Carlos Mendoza en el mes?: 40
✓ Salario de Carlos Mendoza calculado y guardado.
```

```
Seleccione una opción (1-8): 2

--- SECCIÓN DE CÁLCULO DE SALARIOS ---
1. Calcular Empleado de Planta
2. Calcular Empleado Por Hora
Seleccione el tipo de cálculo (1 o 2): 2

Seleccione el empleado Por Hora a calcular:
1. Ana Gómez (Supervisor)
2. María López (Operario)
3. Juan Pérez (Gerente)
4. Sofía Castro (Operario)
5. Lucas Silva (Operario)
Número de empleado: 1
¿Cuántas horas totales trabajó Ana Gómez en el mes?: 40
✓ Salario de Ana Gómez calculado y guardado.
```

II.8.4. Opción 3: Generar Boleta de Pago (Historial Calculado)

- **Operación**: Actúa como el módulo de salida y auditoría del sistema para la emisión de comprobantes fiscales simulados.

- **Comportamiento:** Permite segmentar la búsqueda por tipo de contrato (Planta o Por Hora). El sistema filtra únicamente a los empleados a quienes ya se les procesó el sueldo en la *Opción 2*. Tras seleccionar al trabajador, el software dibuja un cuadro de texto formateado con caracteres especiales que detalla formalmente: Nombre, Cargo, Tipo de Contrato, Registro de Horas/Semanas y el Monto Neto a pagar.

```

Seleccione una opción (1-8): 3

--- REPORTE DE BOLETAS DE PAGO GENERADAS ---
1. Ver boleta de un empleado de Planta
2. Ver boleta de un empleado Por Hora
Seleccione una opción (1 o 2): 2

Seleccione el empleado para ver su boleta:
1. Ana Gómez
Número: 1

=====
          ■ BOLETA DE PAGO - EMPRESA S.A.
=====

Empleado:   Ana Gómez
Cargo:      Supervisor
Contrato:   Por Hora
Registro:   40 horas trabajadas
Total Neto: $3200.00
=====

```

II.8.5. Opción 4: Mostrar Empleado con Mayor Salario (Simulación)

- **Operación:** Ejecuta un análisis predictivo y comparativo sobre toda la nómina en memoria.
- **Comportamiento:** Para estandarizar la comparativa entre contratos de naturaleza distinta, el algoritmo realiza una simulación mensual uniforme (asigna 4 semanas fijas a los empleados de Planta y 160 horas trabajadas a los empleados Por Hora). Utilizando una función de búsqueda de extremos, identifica el ingreso mensual simulado más alto y genera un informe estructurado que desglosa los datos del empleado. En caso de colisión (empates en el salario tope), el sistema imprime un bloque formal para cada uno de los implicados.

```

Seleccione una opción (1-8): 4

=====
🚦 INFORME: MAYOR SALARIO GENERADO (Mensual Estándar)
=====
• Empleado: Carlos Mendoza
  Cargo: Gerente
  Contrato: Planta
  Salario: $16000.00
-----
• Empleado: Juan Pérez
  Cargo: Gerente
  Contrato: Por Hora
  Salario: $16000.00
-----

```

II.8.6. Opción 5: Mostrar Empleado(s) con Mayor Tiempo (Antigüedad)

- **Operación:** Módulo estadístico enfocado en la fidelidad y trayectoria del capital humano dentro de la organización.
- **Comportamiento:** El algoritmo examina la propiedad antigüedad de todos los objetos en la lista, determina cuál es el número máximo de años registrados y realiza un barrido de filtrado. Extrae e imprime un reporte formal con un diseño de ficha técnica que muestra el nombre, cargo, contrato y años de servicio únicamente de aquellos empleados que igualen la antigüedad máxima.

```

Seleccione una opción (1-8): 5

=====
📄 INFORME: EMPLEADOS CON MAYOR ANTIGÜEDAD
=====

• Empleado:    Jorge Rivas
  Cargo:       Supervisor
  Contrato:    Planta
  Antigüedad:  12 años de trabajo

-----

• Empleado:    Luis Torres
  Cargo:       Operario
  Contrato:    Planta
  Antigüedad:  12 años de trabajo

-----

```

II.8.7. Opción 6: Aumentar (Agregar) un Nuevo Empleado

- **Operación:** Implementa la operación **Create (Crear)** del patrón CRUD, permitiendo la expansión dinámica de la base de datos interna.
- **Comportamiento:** Solicita de forma interactiva el nombre completo, el cargo (validando estrictamente que pertenezca a las categorías permitidas) y los años de antigüedad. Finalmente, pide definir el tipo de contratación (1 para Planta, 2 para Por Hora) para instanciar dinámicamente la clase hija correspondiente y anexarla a la lista principal con el método `.append()`.

```

Seleccione una opción (1-8): 6

--- AGREGAR NUEVO EMPLEADO ---
Nombre completo: Vannesa Linda
Cargo (Gerente, Supervisor, Operario): Supervisor
Años de antigüedad: 0
Tipo de empleado (1: Planta / 2: Por Hora): 1
✓ Vannesa Linda agregado con éxito a Planta.

```

II.8.8. Opción 7: Eliminar un Empleado de la Lista

- **Operación:** Implementa la operación **Delete (Eliminar)** del patrón CRUD para depurar la base de datos de la sesión.
- **Comportamiento:** Despliega la nómina completa indexada numéricamente. Tras capturar el índice seleccionado por el usuario, utiliza el método de listas `.pop()` para remover el objeto del arreglo principal. Adicionalmente, el sistema ejecuta una limpieza de dependencias, eliminando cualquier boleta de pago o registro histórico asociado a ese nombre en el diccionario de salarios para evitar corrupción de datos.


```

Seleccione una opción (1-8): 7

--- ELIMINAR EMPLEADO ---
1. Carlos Mendoza (Gerente) - Planta
2. Jorge Rivas (Supervisor) - Planta
3. Luis Torres (Operario) - Planta
4. Elena Rostova (Operario) - Planta
5. Pedro Infante (Supervisor) - Planta
6. Ana Gómez (Supervisor) - Por Hora
7. María López (Operario) - Por Hora
8. Juan Pérez (Gerente) - Por Hora
9. Sofía Castro (Operario) - Por Hora
10. Lucas Silva (Operario) - Por Hora
11. Vannesa Linda (Supervisor) - Planta
Seleccione el número del empleado a eliminar: 11
✗ Vannesa Linda ha sido removido del sistema.

```

II.8.9. Opción 8: Salir del Programa

- **Operación:** Termina de forma segura la ejecución del entorno interactivo.
- **Comportamiento:** Rompe el bucle infinito principal (while True) mediante la instrucción break, despliega un mensaje formal de cierre en la consola y libera los recursos de memoria asignados durante la sesión de trabajo.

```

=====
Selección una opción (1-8): 8

Cerrando el sistema de gestión. ¡Hasta luego!

```

II.9. DESGLOSE TÉCNICO Y ANÁLISIS DE COMPONENTES DEL CÓDIGO

II.9.1. El Núcleo de la Programación Orientada a Objetos (La Clase Padre)

Esta sección del código fuente establece los cimientos de todo el sistema de gestión. Aquí se aplican de forma directa los principios de **Abstracción** y **Encapsulamiento Lógico** mediante la sintaxis nativa de Python.

```
class Empleado:
    """Clase base con las tarifas oficiales."""
    TARIFAS = {"Gerente": 100.0, "Supervisor": 80.0, "Operario": 50.0}

    def __init__(self, nombre: str, cargo: str, antigüedad: int, tipo: str):
        self.nombre = nombre
        self.cargo = cargo.capitalize()
        self.antigüedad = antigüedad # Años en la empresa
        self.tipo = tipo

    def calcular_salario_final(self, factor: int) -> float:
        pass

    def __str__(self) -> str:
        return f"[{self.tipo}] {self.nombre:<20} | Antigüedad: {self.antigüedad}"
```

Explicación de Variables y Componentes:

- **Empleado (Clase Padre):** Es la plantilla genérica. Define los datos y comportamientos comunes que compartirá todo el personal de la empresa.
- **TARIFAS (Variable de Clase / Diccionario):** Almacena las constantes de pago por hora oficiales de la pizarra corporativa. Está centralizada aquí para que, si las tarifas cambian en el futuro, solo se deba modificar una sola línea de código en todo el software.
- **__init__ (Constructor de la Clase):** El método especial que inicializa los atributos de cada empleado cuando se crea en la memoria.
- **Variables de Instancia (Atributos):**
 - self.nombre (Tipo str): Almacena el nombre completo del trabajador.
 - self.cargo (Tipo str): Almacena la posición jerárquica. Se le aplica .capitalize() para asegurar que siempre empiece con mayúscula, estandarizando las búsquedas.
 - self.antigüedad (Tipo int): Almacena el tiempo de servicio medido en años enteros.
 - self.tipo (Tipo str): Almacena la modalidad contractual ("Planta" o "Por Hora").
- **calcular_salario_final(self, factor):** No ejecuta ninguna fórmula aquí porque la Clase Padre no sabe cómo cobra cada contrato. Define la firma del método para obligar a las clases hijas a implementarlo.
- **__str__(self) (Método Mágico de Representación):** Modifica la forma en que el objeto se imprime en pantalla. Uso del formateo <20 para obligar al nombre a ocupar exactamente 20 caracteres de espacio, generando columnas perfectas en la consola.

II.9.2. Subclase de Especialización: EmpleadoPlanta

Esta sección del código fuente modela al personal permanente de la empresa. En este bloque se demuestra la implementación de la **Herencia Jerárquica** y la **Sobreescritura de Métodos (Polimorfismo)** para adaptar las reglas de negocio a los empleados con jornada fija estipulada.

A continuación, se presenta el fragmento de código analizado seguido de su auditoría técnica y estructural:

```
class EmpleadoPlanta(Empleado):
    """Empleados con contrato fijo de 40 horas semanales."""
    def __init__(self, nombre: str, cargo: str, antiguedad: int):
        super().__init__(nombre, cargo, antiguedad, "Planta")
        self.horas_semanales = 40

    def calcular_salario_final(self, semanas_trabajadas: int) -> float:
        tarifa_hora = self.TARIFAS.get(self.cargo, 50.0)
        salario_semanal = tarifa_hora * self.horas_semanales
        return salario_semanal * semanas_trabajadas
```

A. Declaración de la Subclase e Invocación de Herencia (Líneas 1 a 4)

- **Componente:** `class EmpleadoPlanta(Empleado):`
- **Explicación:** La sintaxis de Python indica que `EmpleadoPlanta` es una clase hija que hereda directamente de la superclase `Empleado`. Al hacer esto, adquiere automáticamente todas sus propiedades, constantes y comportamientos sin necesidad de duplicar código.
- **Mecanismo `super().__init__()` (Línea 4):** Invoca al constructor de la clase padre `Empleado` para inicializar las variables comunes (`nombre`, `cargo`, `antiguedad`). Pasa de manera explícita y automatizada la cadena "Planta" en el parámetro del tipo de contrato, liberando al usuario de ingresar este dato manualmente y reduciendo errores operativos.

B. Encapsulamiento de Constantes Contractuales (Línea 5)

- **Variable de Instancia:** `self.horas_semanales = 40`
- **Explicación:** Es un atributo específico y exclusivo de esta subclase. Modela el requisito operacional dictado en la pizarra del negocio: todos los empleados de planta están sujetos a una jornada fija inalterable de 40 horas semanales. Al quedar encapsulado dentro del constructor, queda protegido y disponible para cualquier operación matemática interna de la clase.

C. Implementación Polimórfica y Regla de Negocio Dinámica (Líneas 7 a 10)

- **Método:** `def calcular_salario_final(self, semanas_trabajadas: int) -> float:`
- **Explicación:** Este bloque representa la materialización del **Polimorfismo por Sobreescritura** (Method Overriding). Reemplaza el método simulado con `pass` de la clase padre por una función operativa real. En este contexto contractual

específico, el parámetro genérico *factor* se especializa e interpreta estrictamente como el número de **semanas trabajadas** en el periodo de nómina.

D. Extracción Segura de Datos y Proceso Matemático (Líneas 8 a 10)

- **Variables Locales:**
 - *tarifa_hora*: Obtiene el costo del tiempo laboral mediante la instrucción `self.TARIFAS.get(self.cargo, 50.0)`. Este método interroga al diccionario estático heredado de la clase padre buscando la coincidencia del puesto actual (Gerente, Supervisor, Operario). El valor 50.0 actúa como un mecanismo de mitigación de errores (fallback value); si se ingresó un cargo no parametrizado, el sistema asigna el sueldo base de un operario por defecto, impidiendo que el hilo del programa sufra un colapso crítico por falta de llaves.
 - *salario_semanal*: Variable temporal que calcula el subtotal económico multiplicando la *tarifa_hora* extraída por las 40 horas fijas contractuales obligatorias.
- **Instrucción return (Línea 10)**: Consolida la liquidación final devolviendo el producto del *salario_semanal* por el volumen de *semanas_trabajadas* ingresadas de forma interactiva por el operador.

II.9.3. Subclase de Especialización: EmpleadoPorHora

Esta sección del código modela al personal eventual o temporal. Completa el ciclo de **Herencia** del sistema y consolida el **Polimorfismo** al resolver el mismo cálculo de salario mediante una lógica de jornada flexible.

```
class EmpleadoPorHora(Empleado):
    """Empleados con jornadas flexibles por horas."""
    def __init__(self, nombre: str, cargo: str, antigüedad: int):
        super().__init__(nombre, cargo, antigüedad, "Por Hora")

    def calcular_salario_final(self, horas_totales: int) -> float:
        tarifa_hora = self.TARIFAS.get(self.cargo, 50.0)
        return tarifa_hora * horas_totales
```

A. Herencia Explícita (Líneas 1 a 4)

- **EmpleadoPorHora(Empleado)**: Declara que la clase hereda directamente de la clase padre.
- **super().__init__()**: Reutiliza el constructor del padre para asignar el nombre, cargo y antigüedad de forma automática, inyectando de manera fija el tipo de contrato "Por Hora".

B. Sobreescritura Polimórfica (Líneas 6 a 8)

- **calcular_salario_final**: Reescribe el método abstracto de la clase base. Aquí el parámetro se interpreta de forma exclusiva como **horas totales trabajadas** en el mes, rompiendo la estructura rígida de *semanas* que tenía la clase de Planta.

C. Lógica de Liquidación Dinámica (Línea 7 y 8)

- ***self.TARIFAS.get(self.cargo, 50.0):** Busca en el diccionario heredado el pago asignado al puesto (Gerente, Supervisor, Operario). Si no encuentra el cargo, aplica la tarifa base de \$50.0.*
- ***return tarifa_hora * horas_totales:** Multiplica directamente el valor de la hora por las horas acumuladas ingresadas en el menú, devolviendo el sueldo neto de forma instantánea.*

II.9.4. Inicialización de Estructuras y Persistencia en Memoria

Esta sección del código inicializa las estructuras de datos que servirán como base de datos volátil durante la ejecución del programa. Se definen la plantilla predeterminada de la empresa y el repositorio para las transacciones financieras.

```
empleados = [
    EmpleadoPlanta("Carlos Mendoza", "Gerente", 8),
    EmpleadoPlanta("Jorge Rivas", "Supervisor", 12),
    EmpleadoPlanta("Luis Torres", "Operario", 12),
    EmpleadoPlanta("Elena Rostova", "Operario", 3),
    EmpleadoPlanta("Pedro Infante", "Supervisor", 5),

    EmpleadoPorHora("Ana Gómez", "Supervisor", 4),
    EmpleadoPorHora("María López", "Operario", 2),
    EmpleadoPorHora("Juan Pérez", "Gerente", 1),
    EmpleadoPorHora("Sofía Castro", "Operario", 6),
    EmpleadoPorHora("Lucas Silva", "Operario", 2)
]

historial_salarios = {}
```

A. Arreglo Dinámico Unificado (Variable empleados)

- **Componente:** empleados (Tipo: list)
- **Explicación:** Es una lista dinámica que almacena de forma conjunta objetos de distintas subclases (EmpleadoPlanta y EmpleadoPorHora). Esto demuestra la capacidad del sistema para manejar colecciones homogéneas en su raíz (Empleado), permitiendo que operaciones CRUD posteriores (como agregar con .append() o eliminar con .pop()) alteren el tamaño de la plantilla en tiempo real.
- **Cumplimiento del Requerimiento:** Cumple estrictamente la directiva de la pizarra al incorporar exactamente **5 empleados de cada tipo contractual** para totalizar una nómina inicial de 10 trabajadores registrados con diferentes puestos y antigüedades.

B. Repositorio de Auditoría Financiera (Variable historial_salarios)

- **Variable:** historial_salarios (Tipo: dict)

- **Explicación:** Diccionario vacío inicializado en el ámbito global. Funciona como la base de datos en memoria para el almacenamiento indexado de las liquidaciones que el operador procesa en el menú.
- **Propósito en el Negocio:** Almacena estructuras anidadas que contienen el cargo, tipo de contrato, detalle de tiempo y monto neto calculado. Esto permite que la opción de emitir boletas consulte instantáneamente este mapa mediante el nombre del empleado, garantizando eficiencia de acceso y evitando la pérdida de información durante la sesión.

II.9.5. Módulo de Control de Interfaz: Inicialización del Menú

Esta sección del código fuente establece el entorno interactivo del software. Inicializa la consola de comandos principal mediante una estructura de ciclo infinito, garantizando la persistencia operativa y la captura controlada de directivas de usuario.

```
def menu():
    cargos_orden = ["Gerente", "Supervisor", "Operario"]

    while True:
        print("\n" + "="*50)
        print("SISTEMA DE GESTIÓN DE EMPLEADOS.")
        print("="*50)
        print("1. 📋 MOSTRAR ESCALA DE EMPLEADOS")
        print("2. 💰 CALCULAR SALARIO DE EMPLEABILIDAD")
        print("3. 📄 GENERAR BOLETA DE PAGO")
        print("4. 🏠 MOSTRAR EMPLEADO CON MAYOR SALARIO")
        print("5. ⌚ MOSTRAR EMPLEADOS CON MAYOR TIEMPO LABORAL")
        print("6. ➕ AÑADIR NUEVO EMPLEADO")
        print("7. ✖ ELIMINAR EMPLEADO DE LA EMPRESA")
        print("8. 🚪 SALIR DEL PROGRAMA")
        print("="*50)

        opcion = input("Seleccione una opción (1-8): ").strip()
```

A. Definición de Criterio Jerárquico Corporativo

- **Variable:** cargos_orden (Tipo: list)
- **Explicación:** Arreglo estático de cadenas de texto que define de forma explícita el orden de importancia de los puestos organizacionales de la empresa. Sirve como matriz indexada de referencia para estructurar por prioridades descendentes las consultas y listados subsecuentes.

B. Persistencia Operativa en la Interfaz (Bucle Principal)

- **Componente:** while True:
- **Explicación:** Bloque de control de flujo cíclico indeterminado (bucle infinito). Su propósito técnico es mantener la aplicación en estado de escucha permanente,

asegurando que el programa no finalice tras resolver una tarea y permitiendo realizar múltiples consultas durante la misma sesión.

C. Captura y Sanitización de Comandos de Usuario

- **Variable / Componente:** `opcion = input(...).strip()`
- **Explicación:** Captura la entrada tipográfica ingresada por teclado. La incorporación del método nativo `.strip()` al final de la línea es un mecanismo crítico de robustez: elimina automáticamente cualquier espacio en blanco accidental que el operador introduzca al inicio o al final de su respuesta (por ejemplo, transformar " 2 " en "2"), mitigando errores de coincidencia en el posterior árbol condicional de selección.

II.9.6. Módulo Analítico Organizacional: Opción 1 (Escala Jerárquica)

Esta sección ejecuta el mapeo ordenado de la empresa. Utiliza algoritmos de filtrado por coincidencia de cadenas y discriminación por tipo de instancia para clasificar dinámicamente al personal según su rango y su tipo de contrato.

```
if opcion == "1":
    print("\n=====")
    print("          ESTRUCTURA JERÁRQUICA DE LA EMPRESA          ")
    print("=====")
    for cargo in cargos_orden:
        print(f"\n ♦ SECCIÓN: {cargo.upper()}S")
        print("-" * 40)

        print("\n [CONTRATO DE PLANTA]")
        empleados_planta = [e for e in empleados if e.cargo == cargo and isinstance(e, EmpleadoPlanta)]
        if empleados_planta:
            for emp in empleados_planta:
                print(f"    • {emp.nombre:<20} | Antigüedad: {emp.antigüedad} años")
        else:
            print("    (No hay empleados de Planta en este cargo)")

        print("\n [CONTRATO POR HORA]")
        empleados_hora = [e for e in empleados if e.cargo == cargo and isinstance(e, EmpleadoPorHora)]
        if empleados_hora:
            for emp in empleados_hora:
                print(f"    • {emp.nombre:<20} | Antigüedad: {emp.antigüedad} años")
        else:
            print("    (No hay empleados Por Hora en este cargo)")
        print("-" * 40)
```

A. Iteración Jerárquica Indexada (Bucle for cargo)

- **Componente:** `for cargo in cargos_orden:`
- **Explicación:** El bucle principal lee consecutivamente los valores de la lista de prioridad establecida ("Gerente", "Supervisor", "Operario"). Esto garantiza que el reporte en pantalla siempre respete una subordinación ejecutiva lógica descendente, independientemente del orden de inserción de los empleados.

B. Comprensiones de Lista con Discriminación de Tipo (isinstance)

- ***Variables:*** *empleados_planta* y *empleados_hora* (Tipo: list)
- ***Explicación:*** *Son colecciones secundarias que filtran la base de datos principal en una sola línea mediante List Comprehensions. Implementan la función nativa isinstance(objeto, Clase). Este es un mecanismo clave de la POO: evalúa en tiempo de ejecución a qué clase hija pertenece el objeto, permitiendo separar a los trabajadores fijos de los variables que ocupan un mismo puesto.*

C. Control de Flujo Adaptativo ante Escenarios Vacíos (if-else)

- ***Componente:*** *if empleados_planta: / else:*
- ***Explicación:*** *Evaluación booleana que verifica si la lista filtrada posee elementos. Si está vacía, el bloque else inyecta un mensaje informativo en lugar de dejar el espacio en blanco o romper el formato, asegurando la robustez e integridad visual del informe impreso en la consola.*

II.9.7. Módulo de Procesamiento de Nómina: Opción 2 (Cálculo de Salarios)

Esta sección administra la liquidación individual de sueldos en memoria. Aplica técnicas de indexación dinámica para la selección de personal, inyecta las variables temporales a los métodos polimórficos de las subclases y registra los resultados en el historial interno.


```

elif opcion == "2":
    print("\n--- SECCIÓN DE CÁLCULO DE SALARIOS ---")
    print("1. Calcular Empleado de Planta.")
    print("2. Calcular Empleado Por Hora.")
    sub_opcion = input("Seleccione el tipo de cálculo (1 o 2): ").strip()

    if sub_opcion == "1":
        plantas = [e for e in empleados if isinstance(e, EmpleadoPlanta)]
        if not plantas:
            print("No hay empleados de planta registrados.")
            continue

        print("\nSeleccione el empleado de Planta a calcular:")
        for idx, emp in enumerate(plantas, 1):
            print(f"{idx}. {emp.nombre} ({emp.cargo})")
        try:
            seleccion = int(input("Número de empleado: ")) - 1
            if 0 <= seleccion < len(plantas):
                emp_elegido = plantas[seleccion]
                semanas = int(input(f"¿Cuántas semanas trabajó {emp_elegido.nombre} en el mes?: "))

                salario = emp_elegido.calcular_salario_final(semanas)
                historial_salarios[emp_elegido.nombre] = {
                    "cargo": emp_elegido.cargo,
                    "tipo": emp_elegido.tipo,
                    "detalle": f"{semanas} semanas trabajadas",
                    "monto": salario
                }
                print(f"✓ Salario de {emp_elegido.nombre} calculado y guardado.")
            else:
                print("✗ Número fuera de rango.")
        except ValueError:
            print("✗ Entrada inválida. Use números enteros.")

    elif sub_opcion == "2":
        por_horas = [e for e in empleados if isinstance(e, EmpleadoPorHora)]
        if not por_horas:
            print("No hay empleados por hora registrados.")
            continue

        print("\nSeleccione el empleado Por Hora a calcular:")
        for idx, emp in enumerate(por_horas, 1):
            print(f"{idx}. {emp.nombre} ({emp.cargo})")
        try:
            seleccion = int(input("Número de empleado: ")) - 1
            if 0 <= seleccion < len(por_horas):
                emp_elegido = por_horas[seleccion]
                horas = int(input(f"¿Cuántas horas totales trabajó {emp_elegido.nombre} en el mes?: "))

                salario = emp_elegido.calcular_salario_final(horas)
                historial_salarios[emp_elegido.nombre] = {
                    "cargo": emp_elegido.cargo,
                    "tipo": emp_elegido.tipo,
                    "detalle": f"{horas} horas trabajadas",
                    "monto": salario
                }
                print(f"✓ Salario de {emp_elegido.nombre} calculado y guardado.")
            else:
                print("✗ Número fuera de rango.")
        except ValueError:
            print("✗ Entrada inválida. Use números enteros.")

    else:
        print("✗ Opción incorrecta.")

```

A. Enumeración Dinámica e Indexación de Selección

- **Componente:** `for idx, emp in enumerate(coleccion, 1):` y `seleccion = int(...) - 1`
- **Explicación:** El sistema utiliza la función `enumerate(..., 1)` para imprimir en la terminal un listado numerado a partir del número 1 (legible para el usuario humano). Inmediatamente después, el programa resta una unidad (- 1) al valor capturado por teclado para realizar el mapeo de arreglos correspondiente dentro del índice base 0 real de la lista en Python.

B. Control Estructurado de Excepciones y Rangos (`try-except` / `if`)

- **Componente:** `try: ... except ValueError: e if 0 <= seleccion < len(...):`
- **Explicación:** Mecanismo de doble validación para robustez del software. El bloque condicional `if` verifica que el número elegido exista dentro de los límites físicos del arreglo actual. Paralelamente, el bloque `except ValueError` intercepta cualquier error de conversión tipográfica si el operador digita caracteres alfabéticos o símbolos en lugar de valores numéricos, evitando un colapso crítico del programa.

C. Invocación Polimórfica y Mutación de Historial

- **Variables:** `salario` (Tipo: `float`) e `historial_salarios` (Tipo: `dict`)
- **Explicación:** El puntero `emp_elegido` ejecuta el método polimórfico `calcular_salario_final()`. Python detecta automáticamente a qué clase hija pertenece el objeto seleccionado y aplica la fórmula respectiva (semanas o acumulado de horas). Tras la resolución matemática, el sistema inyecta un diccionario anidado dentro del mapa global `historial_salarios`, indexándolo con el nombre del empleado como llave única para asegurar la persistencia transaccional de la sesión.

II.9.8. Módulo de Consultas y Auditoría: Opción 3 (Generación de Boletas de Pago)

Esta sección del programa actúa como la capa de salida de datos para la emisión de comprobantes financieros. Valida la preexistencia de transacciones en la memoria volátil, extrae las estructuras de datos anidadas del historial y las imprime bajo un formato estructurado de recibo ejecutivo.

```
elif opcion == "3":
    print("\n--- REPORTE DE BOLETAS DE PAGO GENERADAS ---")
    print("1. Ver boleta de un empleado de Planta")
    print("2. Ver boleta de un empleado Por Hora")
    sub_boleta = input("Seleccione una opción (1 o 2): ").strip()

    if sub_boleta == "1":
        plantas_calculados = [e for e in empleados if isinstance(e, EmpleadoPlanta) and e.nombre in historial_salarios]
        if not plantas_calculados:
            print("⚠ No se han calculado salarios para empleados de Planta aún.")
            continue
        print("\nSeleccione el empleado para ver su boleta:")
        for idx, emp in enumerate(plantas_calculados, 1):
            print(f"{idx}. {emp.nombre}")
        try:
            sel = int(input("Número: ")) - 1
            if 0 <= sel < len(plantas_calculados):
                emp_sel = plantas_calculados[sel]
                datos = historial_salarios[emp_sel.nombre]
                print("\n" + "-"*45)
                print(f"    ■ BOLETA DE PAGO - EMPRESA S.A.")
                print("-"*45)
                print(f"Empleado:      {emp_sel.nombre}")
                print(f"Cargo:         {datos['cargo']}")
                print(f"Contrato:      {datos['tipo']}")
                print(f"Registro:     {datos['detalle']}")
                print(f"Total Neto:   ${datos['monto']:.2f}")
                print("-"*45)
            else:
                print("❌ Número fuera de rango.")
        except ValueError:
            print("❌ Entrada inválida.")
```

```

elif sub_boleta == "2":
    horas_calculados = [e for e in empleados if isinstance(e, EmpleadoPorHora) and e.nombre in historial_salarios]
    if not horas_calculados:
        print("⚠ No se han calculado salarios para empleados Por Hora aún.")
        continue
    print("\nSeleccione el empleado para ver su boleta:")
    for idx, emp in enumerate(horas_calculados, 1):
        print(f"{idx}. {emp.nombre}")
    try:
        sel = int(input("Número: ")) - 1
        if 0 <= sel < len(horas_calculados):
            emp_sel = horas_calculados[sel]
            datos = historial_salarios[emp_sel.nombre]
            print("\n" + "="*45)
            print(f"    BOLETA DE PAGO - EMPRESA S.A.")
            print("="*45)
            print(f" Empleado:    {emp_sel.nombre}")
            print(f" Cargo:      {datos['cargo']}")
            print(f" Contrato:   {datos['tipo']}")
            print(f" Registro:   {datos['detalle']}")
            print(f" Total Neto:  ${datos['monto']:.2f}")
            print("="*45)
        else:
            print("❌ Número fuera de rango.")
    except ValueError:
        print("❌ Entrada inválida.")
    else:
        print("❌ Opción inválida.")

```

A. Filtrado por Intersección de Datos (and ... in ...)

- **Componentes:** `plantas_calculados` y `horas_calculados` (Tipo: list)
- **Explicación:** Técnica avanzada de filtrado mediante comprensiones de lista que evalúa dos condiciones simultáneas. Discrimina los objetos de la plantilla según su subclase específica (`EmpleadoPlanta` o `EmpleadoPorHora`) y, al mismo tiempo, verifica si su propiedad `e.nombre` existe como llave dentro del mapa `historial_salarios`. Esto garantiza que solo aparezcan en el menú los empleados que ya tienen un sueldo liquidado, evitando búsquedas fallidas.

B. Consulta Indexada de Estructuras Anidadas (Variable datos)

- **Variable:** `datos` (Tipo: dict)
- **Explicación:** Puntero local que recupera el sub-diccionario guardado en la memoria transaccional correspondiente al trabajador seleccionado (`historial_salarios[emp_sel.nombre]`). Permite acceder a las propiedades internas guardadas en la Fase VII mediante la sintaxis de corchetes e índices de cadena (`datos['cargo']`, `datos['monto']`).

C. Formateo y Máscara de Salida Financiera

- **Componente:** `print(f" Total Neto: ${datos['monto']:.2f}")`
- **Explicación:** Inyección de variables con formato de salida controlado. La especificación técnica `:.2f` actúa como una máscara que redondea y restringe la visualización del salario numérico a exactamente dos dígitos decimales, garantizando un diseño visual limpio y un estándar de presentación formal para el informe en consola.

II.9.9. Módulo de Análisis Predictivo: Opción 4 (Mayor Salario)

Esta sección ejecuta una simulación estadística para homologar las diferencias contractuales de la nómina. Calcula de forma proyectiva los ingresos mensuales estándar,

aisla el extremo superior numérico mediante una expresión generadora y emite fichas técnicas individuales controlando los empates en el salario tope.

```
elif opcion == "4":
    if not empleados:
        print("✗ No hay empleados en la lista.")
        continue
    print("\n" + "="*55)
    print(" 📄 INFORME: MAYOR SALARIO GENERADO (Mensual Estándar)")
    print("="*55)

    salarios_simulados = []
    for emp in empleados:
        sal = emp.calcular_salario_final(4) if isinstance(emp, EmpleadoPlanta) else emp.calcular_salario_final(160)
        salarios_simulados.append((sal, emp))

    # Buscamos el salario máximo analizando únicamente los números
    max_valor_salario = max(sal for sal, emp in salarios_simulados)

    # Filtramos e imprimimos comparando solo los números para evitar colisiones
    for sal, emp in salarios_simulados:
        if sal == max_valor_salario:
            print(f" • Empleado: {emp.nombre}")
            print(f"   Cargo: {emp.cargo}")
            print(f"   Contrato: {emp.tipo}")
            print(f"   Salario: ${sal:.2f}")
            print("-" * 45)
```

A. Homologación Contractual mediante Operador Ternario

- **Lógica del Bucle:** `sal = emp.calcular_salario_final(4) if isinstance(...) else ...`
- **Explicación:** Para realizar una comparación salarial justa, el algoritmo simula un mes contable estándar. Utiliza una estructura condicional en línea (operador ternario) que evalúa dinámicamente la clase del objeto: si el empleado pertenece a `EmpleadoPlanta`, le inyecta un factor de 4 semanas laborales fijas; si es de tipo `EmpleadoPorHora`, proyecta la jornada estándar mensual asignando 160 horas trabajadas.

B. Prevención Dinámica de Errores de Tipo (Variable `max_valor_salario`)

- **Componente:** `max_valor_salario = max(sal for sal, emp in salarios_simulados)`
- **Explicación:** El sistema procesa colecciones que albergan tuplas compuestas por un valor decimal y un objeto complejo (`(float, Objeto)`). Esta instrucción utiliza una expresión generadora interna para extraer **exclusivamente los valores monetarios numéricos** al momento de computar la función de extremos `max()`. Esto evita que Python intente comparar las clases hijas entre sí cuando hay empates numéricos, eliminando de raíz cualquier excepción de tipo `TypeError`.

C. Algoritmo de Coincidencias para Control de Empates (Bucle de Filtrado)

- **Lógica de Impresión:** `if sal == max_valor_salario:`
- **Explicación:** En lugar de interrumpir la búsqueda tras localizar el primer resultado, el programa realiza un barrido secundario de comparación lineal. Al validar la igualdad matemática contra el tope monetario previamente aislado, el sistema es capaz de imprimir un bloque informativo independiente para cada empleado implicado en el máximo común, resolviendo de forma limpia el requerimiento del negocio ante colisiones de nómina (como el empate de los Gerentes a \$16,000.00).

II.9.10. Módulo Analítico de Fidelidad: Opción 5 (Mayor Antigüedad)

Esta sección ejecuta un análisis retrospectivo sobre la trayectoria laboral de la plantilla en memoria. Determina el tope máximo de años de servicio mediante una expresión generadora y extrae fichas técnicas estructuradas para reconocer la

permanencia institucional, controlando de forma exacta los escenarios de empates estadísticos.

```
elif opcion == "5":
    if not empleados:
        print("✗ No hay empleados en la lista.")
        continue
    max_antigüedad = max(e.antigüedad for e in empleados)
    print("\n" + "=" * 55)
    print(f" 📄 INFORME: EMPLEADOS CON MAYOR ANTIGÜEDAD")
    print("=" * 55)
    for emp in empleados:
        if emp.antigüedad == max_antigüedad:
            print(f" • Empleado:    {emp.nombre}")
            print(f"    Cargo:      {emp.cargo}")
            print(f"    Contrato:   {emp.tipo}")
            print(f"    Antigüedad: {emp.antigüedad} años de trabajo")
            print("-" * 45)
```

A. Aislamiento de Extremos Numéricos Secuenciales

- **Variable:** `max_antigüedad` (Tipo: `int`)
- **Explicación:** La instrucción `max(e.antigüedad for e in empleados)` utiliza una expresión comprensiva para mapear y evaluar exclusivamente la propiedad entera de años de servicio de toda la colección. Al aislar este límite matemático de forma independiente, el programa asegura que la posterior búsqueda de coincidencias trabaje sobre un valor numérico precalculado en memoria estática.

B. Algoritmo Iterativo de Barrido Lineal (Bucle de Filtrado)

- **Lógica Condicional:** `if emp.antigüedad == max_antigüedad:`
- **Explicación:** En lugar de interrumpir la ejecución tras localizar el primer registro coincidente, el bucle `for` realiza una comparación lineal contra toda la nómina de trabajadores. Esto garantiza la equidad del reporte analítico: si existen múltiples colaboradores con el mismo récord de antigüedad, el sistema emite un bloque formal para cada uno de ellos (como el empate de Jorge Rivas y Luis Torres con 12 años), resolviendo de raíz las colisiones de datos comunes.

C. Remoción de Datos Redundantes y Estructuración de Fichas

- **Diseño Visual:** Remoción del campo "Jornada" e inclusión explícita del campo "Antigüedad" en la sección de salida de la ficha técnica.
- **Explicación:** El formateo de cadenas unifica la estética del reporte con las boletas de pago previas. Desglosa los metadatos esenciales en una matriz de texto indentada verticalmente por columnas, mejorando significativamente la legibilidad del reporte en la interfaz de usuario de consola.

II.9.11. Módulo de Altas Dinámicas: Opción 6 (Agregar Nuevo Empleado)

Esta sección implementa la operación de creación (Create) dentro del patrón CRUD. Solicita interactivamente los metadatos del nuevo trabajador, aplica filtros de validación de dominios, intercepta anomalías de tipo en la entrada y efectúa una instanciación dinámica enlazando el objeto al arreglo principal.

```

elif opcion == "6":
    print("\n--- AGREGAR NUEVO EMPLEADO ---")
    nombre = input("Nombre completo: ").strip()
    cargo = input("Cargo (Gerente, Supervisor, Operario): ").strip().capitalize()
    if cargo not in cargos_orden:
        print("✗ Cargo no válido. Operación cancelada.")
        continue
    try:
        antigüedad = int(input("Años de antigüedad: "))
    except ValueError:
        print("✗ La antigüedad debe ser un número entero. Cancelado.")
        continue

    tipo_emp = input("Tipo de empleado (1: Planta / 2: Por Hora): ").strip()
    if tipo_emp == "1":
        empleados.append(EmpleadoPlanta(nombre, cargo, antigüedad))
        print(f"✓ {nombre} agregado con éxito a Planta.")
    elif tipo_emp == "2":
        empleados.append(EmpleadoPorHora(nombre, cargo, antigüedad))
        print(f"✓ {nombre} agregado con éxito Por Hora.")
    else:
        print("✗ Opción inválida. Cancelado.")

```

A. Validación de Dominios y Normalización de Entradas

- **Componente:** `if cargo not in cargos_orden:`
- **Explicación:** El método `.capitalize()` normaliza el texto ingresado. Inmediatamente después, el operador condicional `not in` evalúa si la cadena resultante pertenece al conjunto de roles válidos definidos en la matriz de la empresa. Si el valor es incorrecto, la instrucción `continue` interrumpe el bloque y redirige al usuario al menú principal, impidiendo la corrupción de la base de datos con puestos inexistentes.

B. Intercepción de Excepciones por Conversión de Tipo (try-except)

- **Componente:** `try: ... except ValueError:`
- **Explicación:** Control estructurado ante fallos accidentales del operador. La función `int()` exige una cadena puramente numérica para inicializar los años de servicio. Si el usuario ingresa caracteres alfabéticos, Python interrumpe la ejecución normal y transfiere el flujo al bloque `except ValueError:`, desplegando una alerta controlada en lugar de permitir un colapso destructivo de la aplicación.

C. Instanciación Polimórfica Dinámica (`.append()`)

- **Lógica Condicional:** `empleados.append(EmpleadoPlanta(...)) / EmpleadoPorHora(...)`
- **Explicación:** Dependiendo del parámetro `tipo_emp`, el sistema determina cuál de las dos subclases debe inicializarse en la memoria. El método de listas `.append()` inserta el objeto resultante directamente en el arreglo dinámico global. Esto altera la longitud del mapa de nómina en tiempo real, permitiendo que todas las

simulaciones y reportes previos reconozcan inmediatamente al nuevo colaborador.

II.9.12. Módulo de Depuración de Nómina: Opción 7 (Eliminar Empleado)

Esta sección implementa la operación de borrado (Delete) del patrón CRUD. Muestra de forma indexada la nómina activa, evalúa los límites físicos del arreglo para evitar desbordamientos de memoria e implementa una rutina de limpieza en cascada para mantener la integridad de los datos financieros históricos.

```
elif opcion == "7":
    print("\n--- ELIMINAR EMPLEADO ---")
    for i, emp in enumerate(empleados, 1):
        print(f"{i}. {emp.nombre} ({emp.cargo}) - {emp.tipo}")
    try:
        indice = int(input("Seleccione el número del empleado a eliminar: ")) - 1
        if 0 <= indice < len(empleados):
            eliminado = empleados.pop(indice)
            historial_salarios.pop(eliminado.nombre, None)
            print(f"✗ {eliminado.nombre} ha sido removido del sistema.")
        else:
            print("✗ Número fuera de rango.")
    except ValueError:
        print("✗ Entrada inválida.")
```

A. Remoción Física de Objetos en Colecciones (.pop())

- **Componente:** `eliminado = empleados.pop(indice)`
- **Explicación:** Tras validar que el valor de la variable `indice` se encuentre dentro de los límites legales del arreglo ($0 \leq \text{indice} < \text{len}(\text{empleados})$), se invoca el método nativo `.pop()`. Esta instrucción extrae el objeto de la lista principal y reajusta automáticamente el tamaño del vector en memoria, transfiriendo la referencia del elemento removido a la variable local `eliminado`.

B. Rutina de Limpieza en Cascada de Dependencias Financieras

- **Componente:** `historial_salarios.pop(eliminado.nombre, None)`
- **Explicación:** Representa un mecanismo crítico de integridad de software. Al eliminar un trabajador, su recibo de nómina almacenado en la Fase VII queda huérfano. Para evitar fugas de memoria o errores referenciales futuros, el programa interroga al diccionario `historial_salarios` usando la propiedad `.nombre` del objeto extraído. El parámetro `None` actúa como un silenciador de excepciones: si el empleado aún no tenía un salario calculado, la línea se ejecuta de forma segura sin interrumpir el programa.

C. Control de Desbordamiento de Índices (Index Out of Range)

- **Lógica Condicional:** `if 0 <= indice < len(empleados): ... else:`
- **Explicación:** Funciona como un escudo lógico de estabilidad. Evita que Python intente acceder a un espacio de memoria inexistente si el operador ingresa un número erróneo (por ejemplo, digitar el número 15 cuando solo hay 10

trabajadores). El bloque `else` captura de inmediato la anomalía e imprime una alerta, garantizando que el entorno interactivo continúe en ejecución normal.

II.9.13. Módulo de Cierre y Punto de Entrada Principal

Esta sección final del código fuente gestiona la interrupción del bucle interactivo principal y establece el disparador de ejecución estricto que Python utiliza para inicializar el entorno de software.

```
elif opcion == "8":
    print("\nCerrando el sistema de gestión. ¡Hasta luego!")
    break
else:
    print("❌ Opción no válida. Intente de nuevo.")

if __name__ == "__main__":
    menu()
```

A. Interrupción Controlada del Bucle General

- **Componente:** `break` (Línea 3)
- **Explicación:** Sentencia de control que detiene inmediatamente el ciclo indeterminado `while True` inicializado en la Fase V. Al romper el flujo infinito del bucle de manera limpia, el programa continúa con las líneas que estén fuera de la función `menu()`, permitiendo una finalización natural del software sin forzar salidas destructivas en el sistema operativo.

B. Captura de Exclusión por Defecto (Manejo de Respuestas Inválidas)

- **Componente:** `else:` (Línea 4)
- **Explicación:** Funciona como una red de seguridad condicional. Si la entrada del usuario no coincide con ninguna de las opciones válidas (números del 1 al 8), el bloque `else` captura el valor residual, despliega un mensaje de advertencia estructurado y devuelve el control al inicio del ciclo, blindando al sistema contra comandos inexistentes.

C. El Disparador Principal de Ejecución (Main Entry Point)

- **Componente:** `if __name__ == "__main__":`
- **Explicación:** Es el punto de entrada oficial y una de las mejores prácticas más estrictas en la ingeniería de software con Python. Verifica si el archivo está siendo ejecutado directamente por el intérprete como el programa principal. Si es así, la variable especial interna `__name__` adopta de forma automatizada el valor `"__main__"`, activando la invocación de la función `menu()`. Este mecanismo evita que el programa se autoejecute por accidente si el módulo fuera importado desde otra aplicación en un desarrollo a gran escala.

III. CONCLUSIONES

- **En relación al modelado organizacional y la herencia):** Se logró modelar con éxito la estructura contractual de la empresa mediante la implementación de una jerarquía de herencia pura en Python. La clase padre Empleado centralizó de manera eficiente los atributos comunes, mientras que las subclases EmpleadoPlanta y EmpleadoPorHora permitieron especializar el comportamiento de cada contrato, demostrando que la herencia elimina la redundancia de código y optimiza la arquitectura del software.
- **En relación a los algoritmos polimórficos de salarios:** El desarrollo del método `calcular_salario_final` demostró la viabilidad y el poder del polimorfismo en sistemas de nómina corporativa. Gracias a este principio, el sistema fue capaz de invocar una misma función sobre la lista unificada de personal, pero ejecutando ecuaciones matemáticas completamente distintas (fijas por semanas o variables por horas) según la naturaleza del objeto, respetando con precisión milimétrica las tarifas asignadas a las categorías de Gerente, Supervisor y Operario.
- **En relación al análisis de datos y control de empates:** Se optimizó el procesamiento analítico de los recursos humanos mediante algoritmos de filtrado basados en funciones integradas y expresiones lambda. Esto permitió que el sistema no solo localizara salarios tope o máximas antigüedades, sino que controlara de forma matemáticamente exacta los escenarios de empates estadísticos, garantizando un informe transparente que despliega a todos los empleados que cumplen con la condición máxima en lugar de omitir información.
- **En relación a la integridad de datos y control de excepciones):** La implementación de la interfaz de consola acoplada a operaciones dinámicas CRUD garantizó una gestión flexible de la plantilla en tiempo real. Asimismo, el uso estructurado de bloques try-except para capturar la excepción `ValueError` validó de forma robusta las entradas del operador, blindando al sistema contra fallos tipográficos accidentales y asegurando la estabilidad y la integridad de la información almacenada en memoria durante toda la sesión.

REFERENCAS BIBLIOGRÁFICAS

- Joyanes Aguilar, L. (2020). *Programación Orientada a Objetos* (5.^a ed.). McGraw-Hill.
- Martin, R. C. (2018). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- Matthes, E. (2023). *Python Crash Course: A Hands-On, Project-Based Introduction to Programming* (3rd ed.). No Starch Press.
- Python Software Foundation. (2026). *The Python Language Reference Manual* (v3.12). Python Documentation. python.org
- Ramalho, L. (2022). *Fluent Python: Clear, Concise, and Effective Programming* (2nd ed.). O'Reilly Media.
- Sommerville, I. (2019). *Ingeniería de Software* (10.^a ed.). Pearson Educación.
- Sweigart, A. (2021). *Beyond the Basic Stuff with Python: Best Practices for Writing Clean Code*. No Starch Press.
- Weidman, S. (2020). *Deep Learning from Scratch: Building with Python from First Principles*. O'Reilly Media.